AgroSmart – Análisis de Algoritmos y Estructuras de Datos

Módulo: Registro de Cultivos (gestor_cultivos.py y pantalla_cultivos.py)

1. Estructuras que usa el módulo

Estructura	Dónde	Para qué
Lista	`GestorCultivos.lista_cultivos`	Guarda todos los cultivos en el orden en que se muestran.
Tabla hash	`TablaHash` (`estructuras/tabla_hash.py`)	Encuentra un cultivo por su código sin recorrer la lista.
Archivo JSON	`datos_cultivos.json`	Guarda los cultivos para que no se pierdan al cerrar.

Los cultivos de este módulo son la fuente única que usan también las pantallas Riego, Calendario y Clima.

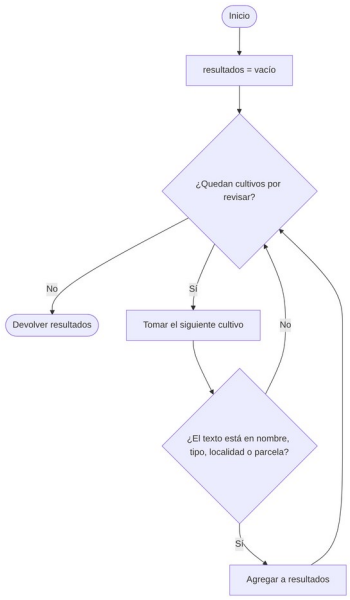
2. Búsqueda lineal

Explicación. Recorre la lista de cultivos uno por uno desde el primero. Hay dos versiones: `buscar_por_nombre` devuelve el primer cultivo cuyo nombre coincide exactamente, y `buscar_coincidencias` devuelve **todos** los cultivos donde el texto aparece en el nombre, el tipo, la localidad o el número de parcela. La pantalla usa la segunda en el botón **Buscar cultivo**.

Pseudocódigo

```
FUNCION buscar_coincidencias(lista, texto)
    texto = MINUSCULAS(texto)
    resultados = []
    PARA CADA cultivo EN lista
        SI texto ESTA EN nombre, tipo, localidad o numero_parcela DEL
cultivo
            AGREGAR cultivo A resultados
        FIN SI
    FIN PARA
    DEVOLVER resultados
FIN FUNCION
```

Diagrama de flujo



Complejidad. Tiempo O(n): en el peor caso revisa todos los cultivos. Espacio: O(1) adicional en `buscar_por_nombre` y O(k) en `buscar_coincidencias` (k = coincidencias).

Justificación. Es sencilla, no necesita que la lista esté ordenada y permite buscar por una parte del texto ("parcela 8", "maíz"). Para la cantidad de cultivos de un agricultor (decenas o cientos) es más que suficiente.

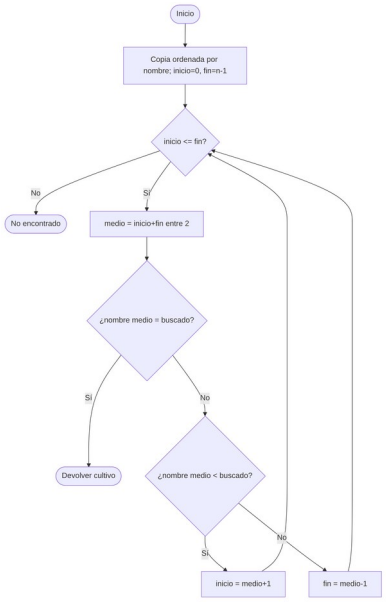
3. Búsqueda binaria

Explicación. Se usa en el botón **Buscar por nombre exacto (rápido)**. Primero crea una copia de la lista ordenada alfabéticamente por nombre (para no desordenar la lista original). Luego revisa el elemento del medio: si coincide, lo devuelve; si el nombre buscado es mayor, descarta la mitad izquierda; si es menor, la derecha. Repite hasta encontrarlo o quedarse sin rango.

Pseudocódigo

```
FUNCION buscar_por_nombre_binaria(lista, nombre)
    ordenada = ORDENAR(lista, por nombre en minúsculas)
    inicio = 0 ; fin = LONGITUD(ordenada) - 1
    MIENTRAS inicio <= fin
        medio = (inicio + fin) DIV 2
        SI ordenada[medio].nombre == nombre: DEVOLVER ordenada[medio]
        SINO SI ordenada[medio].nombre < nombre: inicio = medio + 1
        SINO: fin = medio - 1
    FIN MIENTRAS
    DEVOLVER NADA
FIN FUNCION
```

Diagrama de flujo



Complejidad. Tiempo O(log n) para la búsqueda en sí (más O(n log n) por ordenar la copia). Espacio O(n) por la copia.

Justificación. Permite comparar directamente con la búsqueda lineal: la lineal revisa hasta n cultivos, la binaria necesita como máximo $\log_2 n$ comparaciones. Su desventaja, tener que ordenar, es aceptable con listas de este tamaño.

4. Tabla hash

Explicación. Se implementó una tabla hash propia, con listas dentro de cada casillero para manejar las colisiones. La función hash suma el valor ASCII de cada carácter del código (por ejemplo “CULT001”) y toma el residuo entre el tamaño de la tabla (50). Se usa para buscar un cultivo por código al editar, eliminar o abrirlo desde otra pantalla. Al eliminar un cultivo también se elimina de la tabla.

Pseudocódigo

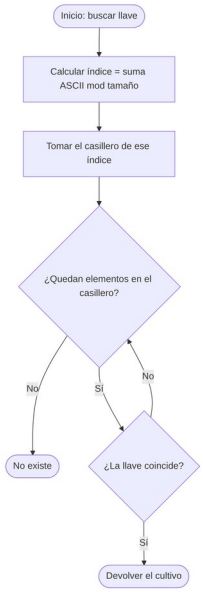
```
FUNCION calcular_indice(llave, tamano)
    suma = SUMA(ASCII(c) PARA CADA c EN llave)
    DEVOLVER suma MODULO tamano

FUNCION insertar(llave, valor)
    casillero = casilleros[calcular_indice(llave)]
    SI llave YA ESTA en casillero: ACTUALIZAR su valor
    SINO: AGREGAR (llave, valor) al casillero

FUNCION buscar(llave)
    PARA CADA (k, v) EN casilleros[calcular_indice(llave)]
        SI k == llave: DEVOLVER v
    DEVOLVER NADA
```

```
FUNCION eliminar(llave)
    QUITAR (llave, valor) del casillero correspondiente
```

Diagrama de flujo (buscar)



Complejidad. Tiempo O(1) en promedio; O(n) en el peor caso si todas las llaves caen en el mismo casillero. Espacio O(n).

Justificación. El código es el identificador único con el que se hacen la mayoría de operaciones. Se programó la tabla propia para poder explicar su funcionamiento interno.

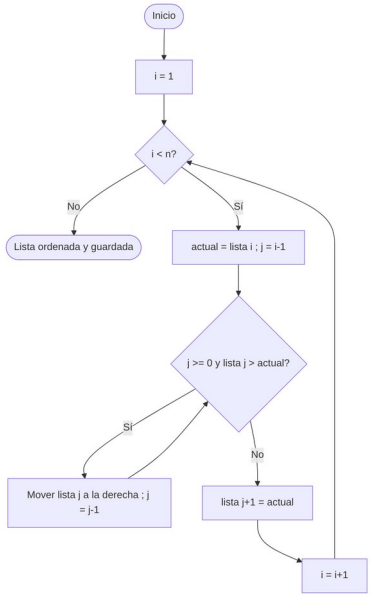
5. Ordenamiento por inserción

Explicación. Ordena los cultivos por fecha de siembra, de la más antigua a la más reciente (botón **Ordenar por fecha de siembra**). Toma un elemento a la vez y lo coloca en su lugar entre los ya ordenados, corriendo los demás una posición a la derecha. Las fechas se guardan como AAAA-MM-DD con ceros, por lo que comparar texto equivale a comparar fechas.

Pseudocódigo

```
FUNCION ordenar_por_fecha_siembra(lista)
    PARA i DESDE 1 HASTA LONGITUD(lista) - 1
        actual = lista[i] ; j = i - 1
        MIENTRAS j >= 0 Y lista[j].fecha_siembra >
actual.fecha_siembra
            lista[j + 1] = lista[j] ; j = j - 1
        FIN MIENTRAS
        lista[j + 1] = actual
    FIN PARA
FIN FUNCION
```

Diagrama de flujo



Complejidad. Tiempo $O(n^2)$ en el peor caso, $O(n)$ si ya está ordenada. Espacio $O(1)$ (ordena en la misma lista).

Justificación. Es simple, estable y muy rápida en listas pequeñas o casi ordenadas, como las de un agricultor. Sirve de contraste con Merge Sort (módulos de Clima y Riego).

6. Conversión de unidades de terreno

Explicación. El agricultor escribe el tamaño en **manzanas, cuerdas o hectáreas**. Por dentro, el sistema guarda siempre **hectáreas** (1 manzana = 0.6987 ha; 1 manzana = 16 cuerdas), para que los módulos de Riego y Planificación calculen con una sola unidad. Al editar, `de_hectareas` convierte de vuelta para mostrar el número y la unidad en el formulario. Si el usuario no cambia el número ni la unidad al editar, se conserva el valor exacto guardado, para que no se "corra" por redondeo.

Pseudocódigo

```
FUNCION a_hectareas(valor, unidad)
    SI unidad == "Hectáreas": DEVOLVER valor
    SI unidad == "Manzanas":  DEVOLVER valor * 0.6987
    SI unidad == "Cuerdas":   DEVOLVER valor * 0.6987 / 16
FIN FUNCION
```

Complejidad. Tiempo $O(1)$, espacio $O(1)$.

7. Validación y normalización de datos

Antes de guardar, `registrar_cultivo` valida: que no haya campos vacíos; que el tamaño sea un número mayor que 0 (`ValueError` si no); y que la fecha sea válida (`normalizar_fecha` la deja con ceros, por ejemplo 2026-10-5 pasa a 2026-10-05). Si algo falla, se muestra un mensaje en rojo y no se guarda nada. Cada código nuevo se genera con un contador que sigue el número más alto ya usado (CULT001, CULT002…), para no repetir códigos aunque se eliminen cultivos.

Complejidad. Validar: O(1). Calcular el contador al cargar: O(n).

8. Guardado y carga (JSON)

Cada operación que cambia los datos (registrar, editar, eliminar, ordenar) reescribe `datos_cultivos.json` (O(n)). Al iniciar, `cargar_datos` lee el archivo, corrige las fechas viejas sin ceros, reconstruye la lista y la tabla hash, y salta los registros incompletos. Si el archivo está dañado, guarda una copia `.bak` y arranca con la lista vacía en lugar de cerrar el programa.